Self Hosting Inventory System Using Smart Cards

Gherab Radouane, Guelouza Abdallah Nadhir, Khier Yassin, Assassi Abderrahim, Neggar Ilyes

I. Introduction

This paper presents a concise proof of concept for a self hosted inventory system based on smart card technologies. It explores the use of MIFARE DESFire for secure data storage and NFC based interaction, alongside a basic implementation using MIFARE Classic 4K for block level read and write operations. The project highlights the contrast between legacy block oriented cards and modern application oriented secure cards in terms of abstraction and security. The goal is not to build a production ready system, but to design, implement, and compare independent academic use cases of DESFire and MIFARE Classic 4K. Each card is used separately to emphasize its architecture, memory organization, and security model. The system operates entirely offline, focusing on practical smart card memory manipulation and stable reader card communication, with clarity and correctness prioritized over performance. MIFARE DESFire was chosen as the primary secure storage medium due to its flexible file system and built in cryptographic support. The card's application structure is leveraged to isolate driver data, mission data, and article records into separate applications. Memory constraints, particularly the 2 KB limit (for EV1) which is later solved using the EV3 (8KB), were addressed through lightweight data compression while preserving essential metadata, resulting in a modular and educational NFC based implementation.

II. Project Overview

The General overview of the proposed system is shown in the Fig 1. Two different smart card technologies are used independently :

- MIFARE DESFire, representing a modern secure smart card with structured memory and advanced security features.
- MIFARE Classic 4K, representing a legacy smart card with sector and block-based memory access.

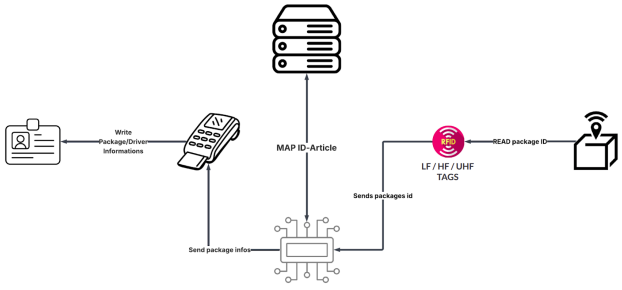

Fig. 1. Conceptual System Architecture

A. MIFARE Classic

For MIFARE Classic cards, the interface performs direct block level write operations after selecting the appropriate block. The Source Interface ensures that all data is serialized into a compact binary format suitable for constrained card memory. This abstraction allows the underlying smart card technology to be used without exposing low level NFC commands to the user.For MIFARE Classic cards, the interface reads raw data directly from the specified blocks and interprets the content based on predefined offsets.

1) MIFARE Classic Access Bits: Access control is managed through a sophisticated permission system defined in the sector trailer. The access bits (C1, C2, C3 for each block trailer) determine which operations require which keys, as illustrated in Fig 2. From Theory to Practice: Implementing Secure Logistics Systems with MIFARE Classic Technology The byte number four in the access control bytes is named GPB (General Purpose Byte) and is reserved for future use (Except only the sector trailer 0 that contain in this field GPB details of the MAD standard)

Operation	C1	C2	C3
Read with Key A	0	0	0
Read with Key B	1	0	0
Write with Key A	0	1	0
Write with Key B	1	1	0
Increment with Key A	0	0	1
Increment with Key B	1	0	1
Decrement/Transfer with Key A	0	1	1
Decrement/Transfer with Key B	1	1	1

Fig. 2. Access Control Bits

As illustrated in Figure 3, the MIFARE Classic card memory is organized in sectors and fixed size blocks, where each block contains 16 bytes. After authenticating to the appropriate sector, the interface performs direct block level write operations by selecting the exact block index defined in the memory layout (e.g., Sector 1 Block 4 for the Application Identifier, Sector 2 Blocks 8–10 for Driver Information, Sector 3 Blocks 12–15 for Mission Data, and Sectors 4–7 Blocks 16–31 for the Articles Inventory). The Source Interface is responsible for serializing all application data into a compact binary representation that strictly fits within the 16 byte block constraint. Each field is formatted according to predefined specifications (e.g., Driver Name), structured patterns for formatted identifiers (e.g., License Plate "AA-123-AA"), numeric encrypted values

for sensitive identifiers (e.g., Driver ID), enumerated values for mission status (0–7), timestamp encoding for temporal data, and multi block Items List (224 bytes across Blocks 16–31). When reading from the card, the interface accesses the same predefined sector and block addresses and retrieves raw 16-byte data chunks. The content is then interpreted using fixed offsets and formatting rules defined by the memory schema shown in the figure. This structured abstraction layer allows the system to reliably use MIFARE Classic cards while hiding low level NFC authentication and block management commands from higher level application logic.

Sector	Block	Purpose	Size	Format
Application Header				
1	4	Application Identifier	16 bytes	"LOGISTICS_V1.0"
Driver Information				
2	8	Driver Name	16 bytes	ASCII, null-terminated
2	9	License Plate	16 bytes	Format: "AA-123-AA"
2	10	Driver ID	16 bytes	Numeric, encrypted
Mission Data				
3	12	Origin Point	16 bytes	Location code
3	13	Destination Point	16 bytes	Location code
3	14	Mission Status	16 bytes	Enum: 0-7
3	15	Timestamp	16 bytes	Unix timestamp
Articles Inventory				
4-7	16-31	Items List	224 bytes	JSON/CSV format

Fig. 3. System Architecture for MIFARE Classic

B. MIFARE DESFire

The proposed use case focuses on a simplified inventory and mission management scenario. DESFire smart card is used to store information related to a driver and an assigned mission. The stored data includes the driver name, license information, driver's compressed image in binary form, mission details and package information. The system is designed to operate in constrained conditions, reflecting realistic scenarios where connectivity may not be available. All data is written to and read from the smart card through an NFC reader and a dedicated application interface.A Desktop application is developed to manage the interaction with the smart card.

- Source interface: The Source Interface represents the data origin point within the system. It is primarily used to initialize and write information to the smart card. This interface is responsible for collecting, formatting, and encoding data before storage on the card. The Source Interface allows the operator to input driver related information such as the driver name, license identifier, mission details, and package metadata. When a MIFARE DESFire card is used, the interface manages application selection, authentication, and structured data writing.
- Destination interface: The Destination Interface is responsible for reading and decoding data stored on the smart card. It represents the verification or inspection point in the system workflow. This interface retrieves stored information and reconstructs it into a human readable format. It reads a card at the delivery point, validate the mission against expected missions, show driver and article details.
- Dashboard, implemented as a web application, responsible for displaying statistics and data visualization.

III. Global System Architecture

A. Operational Workflow

The general operational workflow of the system consists of the following steps:

1) The OMNIKEY reader establishes communication with the smart card.
2) Authentication is performed according to the card's security mechanism.
3) Data is read from or written to the smart card.
4) The response is returned to the application and displayed to the user.

This workflow remains consistent across both smart card technologies, while the internal command sequences differ based on the card architecture.

The overall project is organized into two independent implementations and an API for database and back-end logic, one based on MIFARE DESFire and the other on MIFARE Classic 4K. Each implementation follows a layered structure that separates user interaction, business logic, and hardware communication. This structure allows each component to evolve independently while maintaining a clear and understandable project layout.

```
project/
│
├── .github/workflows
│   └──CI.yaml
├── API
│   ├──CoreAPI
│   ├──EGOV
│   └──manage.py
│
├── MIFARE_DESFire/
│   ├── desfire_library/
│   │   ├── __init__.py
│   │   ├── applications.py
│   │   ├── crypto.py
│   │   ├── desfire_card.py
│   │   ├── files.py
│   │   ├── utils.py
│   │   └── desfire_tests.py
│   │
│   ├── ui/
│   │   ├── __init__.py
│   │   ├── destination_interface.py
│   │   ├── pic_codec.py
│   │   └── source_interface.py
│   │
│   ├── __init__.py
│   ├── main.py
│   └── requirements.txt
```

```
└── MIFARE_CLASSIC_4K/
    ├── data/
    │   └── database.db
    │
    └── logistics_system/
        ├── core/
        │   ├── auth.py
        │   ├── card_manager.py
        │   ├── database.py
        │   └── mission_manager.py
        │
        ├── models/
        │   ├── mission.py
        │   └── user.py
        │
        ├── ui/
        │   ├── admin_panel.py
        │   ├── login_dialog.py
        │   ├── main_window.py
        │   ├── recieve_interface.py
        │   └── send_interface.py
        │
        ├── main.py
        └── requirements.txt
```

B. MIFARE DESFire

The process of sending data between an Omnikey reader and a MIFARE DESFire card follows the ISO/IEC 14443 Type A standard at 13.56 MHz. Initially, the Omnikey reader detects the card through anticollision and selection, obtaining its Unique Identifier (UID). One the selection succssed, the reader authenticate to the PICC master application (ID:000000) using an Authenticate command (e.g., 0x0A with key no. 0x00), establishing a secure session with challenge-response exchange. Next, create a custom application via CreateApplication (0xCA, specifying app ID [000001-Driver App ; 000002-Mission App, 000003-Articles App], key settings, and number of keys). Then, select the custom application with SelectApplication (0x5A followed by the 3-byte app ID) and Authenticate to this custom app using Authenticate (0x0A with the app's key number, e.g., 0x00) to derive a new session key. For every app Create its appropreate file(s) using CreateStdDataFile (0xCD), defining file ID, size, and access rights. Finally, write data to the file with WriteData (0x3D, including file ID, offset, and data payload), after which the card confirms integrity before the session ends.

1) MIFARE DESFire Library: A dedicated helper library was implemented to abstract the complexity of MIFARE DESFire operations, encapsulating tasks such as application selection, authentication, file creation, and secure read/write operations. On the host side, a Python library wraps low-level APDUs (ISO/IEC 14443-4) into high-level classes—`DesfireCard` for communication and authentication, `ApplicationManager` for managing applications, and `FileManager` for handling files—allowing higher-level modules to store and retrieve structured data without directly handling cryptographic keys or raw command frames. Each DESFire command (e.g., `SelectApplication`, `CreateApplication`, `CreateFile`, `ReadData`, `WriteData`) is sent as a `0x90` CLA APDU, with the library transparently handling status codes such as `0x91 0x00` (success) and `0x91 0xAF` (additional frame). This approach improves code readability, reduces implementation errors, and, while not targeting production-level security, reflects the logical workflow of real-world DESFire systems, serving as an effective educational tool.

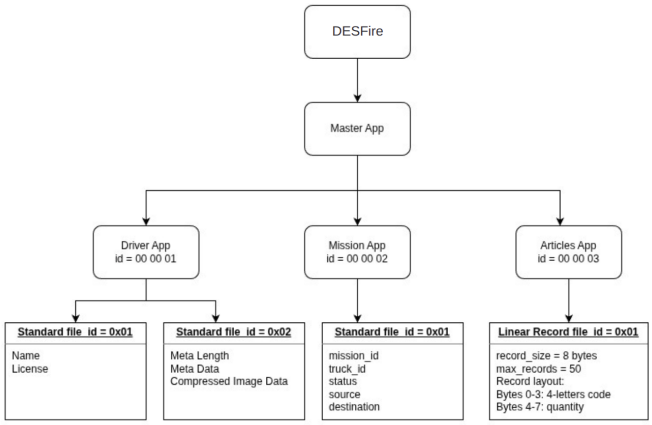

Fig. 4. DESFire EV1/EV3 Application and File Structure

Authentication is performed using the native DES three pass mutual authentication protocol with key number 0 as the master key in each application. The host derives a challenge response using CBC mode DES, rotates the card challenge as required by the specification, and establishes a session key once authentication succeeds. Only after a successful application level authentication are security sensitive operations such as file creation, deletion, or writing allowed, according to the configured key settings As shown in Fig. 5.

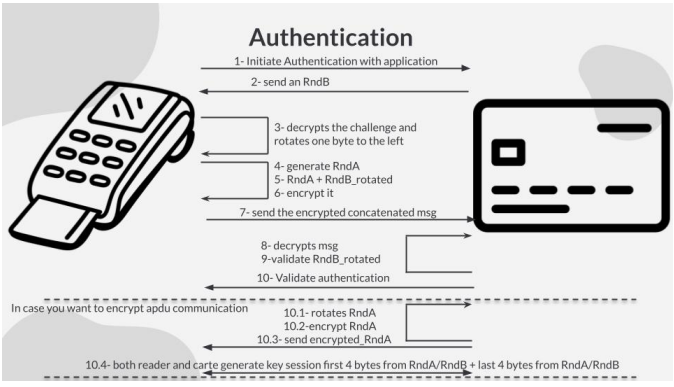

Fig. 5. Authentication process

We created three distinct applications on each card:

- a **driver application** (`AID = 00 00 01`) with stan-

Application ID	File ID	File Type	File Size	Informations
Driver ID=[0x00 0x00 0x01]	0x01	Standard	20 bytes	driver_name + driver_license
	0x02	Standard	1K bytes	[meta_length 4B] + [JSON metadata] + [compressed image data]
Mission ID=[0x00 0x00 0x02]	0x01	Standard	57 bytes	mission_id + truck_id + status + source + destination
Articles ID=[0x00 0x00 0x03]	0x01	Linear Record	record_size = 8 bytes	4-letter code (“LAPT”) + quantity

TABLE I
MIFARE DESFIRE SMART CARD STRUCTURE

dard data files for the driver’s name and license number, plus a large standard file used to store a compressed portrait image.

- a **mission application** (`AID = 00 00 02`) with a standard file of 57 bytes encoding mission identifier, truck identifier, mission status, source, and destination in a fixed offset binary layout.
- an **articles application** (`AID = 00 00 03`) with a linear record file of 8 bytes per record to store article codes (4 ASCII characters) and quantities (32–bit integer) as an append only mission manifest.

Standard data files are used whenever random access and in place updates are required, for example to update only the mission status byte, while linear record files are used for article lists so that each new article is appended as a separate record without modifying previous entries. File access rights are configured so that reading and writing always require authentication with the application master key, while file creation and deletion permissions are controlled through the application key settings byte, allowing the project to enforce that only authenticated terminals can add or remove files.

The system implements a structured approach to storing driver information on the MIFARE DESFire card through a series of application and file management operations as shown in Fig. 8. The process begins with creating the driver application on the card using the create application command, which takes: A three byte Application Identifier (AID) e.g. [0x00, 0x00, 0x01] Key settings (typically 0x0F for default access conditions) The number of keys (usually 0x01 for a single master key). This establishes a secure container for all driver related data within the card’s memory space. Application Creation uses DESFire native command 0xCA to create a new application. It returns success status (0x91 0x00) upon completion. Once the application is created, the system manages two types of data files, one for textual driver information (name and license number) and another for the compressed facial image. Standard data files are created using the create standard file method, which accepts a file identifier (file id), desired file size, communication settings (default 0x00), and access rights (default [0x00, 0x00] for full access). Standard file creation uses the command 0xCD. Data writing to these files is handled by the write data method, which implements low level file access operations. This method constructs APDU commands with the file identifier, three byte offset for positioning within the file, three byte length specification, and the actual data payload. The offset mechanism enables random access within the file, allowing data to be written at specific positions rather than sequentially from the beginning. For driver textual information, the write driver infos method provides a high level interface that concatenates driver name and license strings, encodes them as bytes, and writes them to the designated driver information file starting at offset 0. This approach ensures that all textual driver data is stored contiguously and can be retrieved efficiently during read operations. The most complex operation involves writing the compressed facial image data through the write compressed image method. This function does the following: serializes the metadata (containing shape information) to JSON and encodes it as bytes. A four byte metadata length prefix is prepended to facilitate reconstruction during reading. The complete payload consisting of the length prefix, metadata bytes, and compressed image data is then written to the card in 47 byte chunks. During the write process, the DESFire EV1 card responds to each intermediate chunk with status code 0xAF (Additional Frame) to indicating that it has successfully received the current chunk and additional data is needed. This 0xAF response continues for each chunk until the final data frame is written, at which point the card responds with 0x00 to confirm successful write operation. The function returns total bytes (the offset) written to verify that the full image data has been sent.Fig. 6.

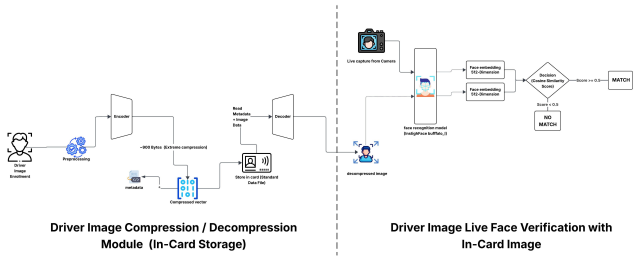

Fig. 6. General process of Compression/Decompression

Since any application and file needs a creation and authentication process to allow any form of read/write operation. The mission manipulation starts with a POST request that sends all relevant fields to the tailor made API. with the values Truck, source and destination, the API from the truc infers the assigned driver, and creates the mission unique ID (e.g

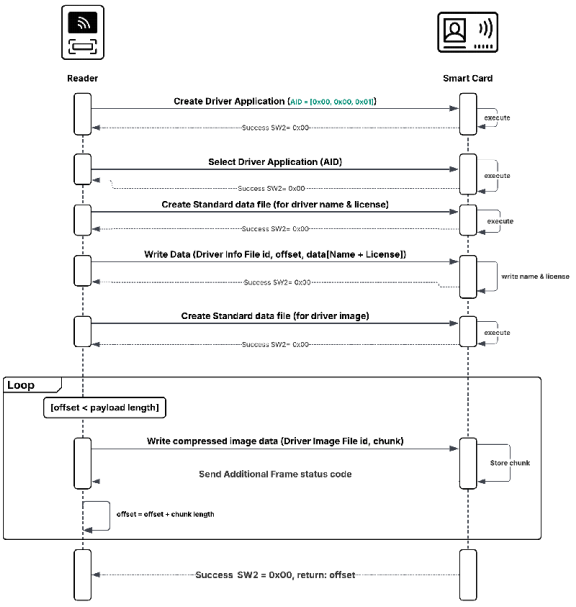

Fig. 7. Write Driver Information

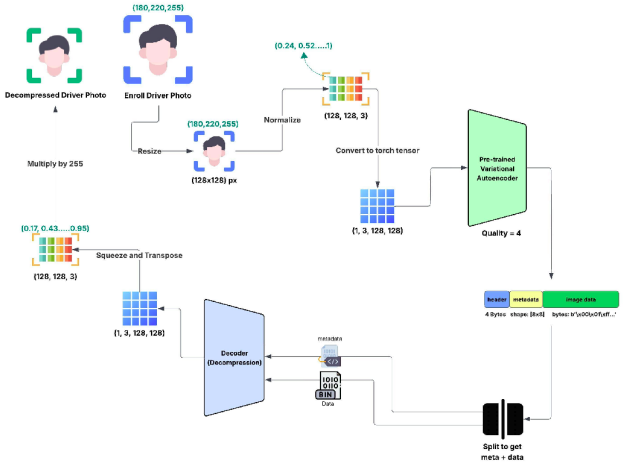

Fig. 8. Detailed Compression/Decompression

MSN00001) and records it on the systems admin dashboard. A valid JSON response is sent back to the Desktop application, based on it the complete data information is concatenated then divided into sizable chunks. The reader is prompted by the application to establish an authenticated connection with the smart card with the targeted application/file id. An APDU using the ISO7816 wrapping standard is sent with the correct command class and instruction tailed with all the data and metadata(data length and expected response ..) to be saved. The smart card saves the data with the correct format to the correct path(in this case the application file combo). Since the file used in this instance is a standard file. no commit is required to finalize the operation. In the same manner the read operation can largely be the same Fig 9.

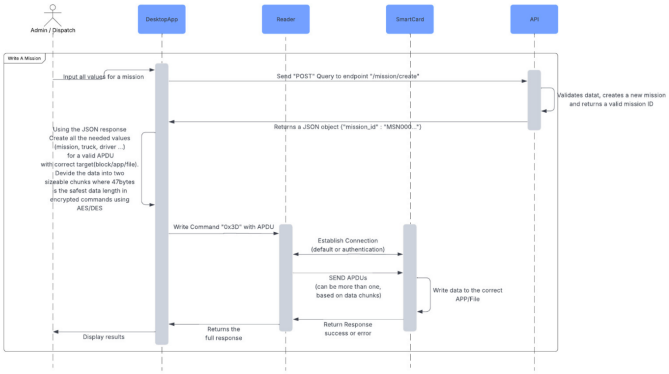

Fig. 9. Sequence of Writing Mission Information

Figure 10 illustrates the complete workflow for writing article data into a MIFARE DESFire smart card using an HF(NFC) RFID reader. The process begins when the RFID reader scans the article tag (ID-Article), then maps this identifier to its corresponding metadata through a backend database. After mapping, the reader communicates with the DESFire card through the RF interface following the ISO/IEC 14443 protocol sequence: Request (REQA), Answer to Request (ATQA), anti-collision and UID selection, and optional HALT. Once the card is selected, an application level secure session is established using the Authenticate command (APDU 0x0A). The system then creates and selects a dedicated application (App3: ID 000003) using APDU commands (0xCA for Create Application and 0x5A for Select Application). After successful authentication within the selected application, a Linear Record file (APDU 0xC1) is created to store structured article data. Finally, the article information retrieved from the database is written into the smart card using the WriteData command (APDU 0x3D) in a loop until all records are stored, with each operation validated by a success response (status 0x91 0x00). This workflow ensures secure, structured, and reliable data storage of mapped article information inside the DESFire smart card.

The resulting architecture demonstrates that DESFire can be used as a self contained secure storage back-end for an inventory system: all mission identifiers, driver identities, and article manifests are stored directly on the card in typed files, protected by symmetric keys and fine grained access rights, while the host application operates through a thin, reusable Python abstraction layer built on top of the native DESFire command set.

C. API

The API side of the project is designed to complement the functioning of the SmartCard implementation, with a strict validation of all the data that needs processing. It was created in a way to allow scalability and delegate the bulk of the data handling to a more reliable entity, leaving the core functions to application's logic. The API allows for the management in

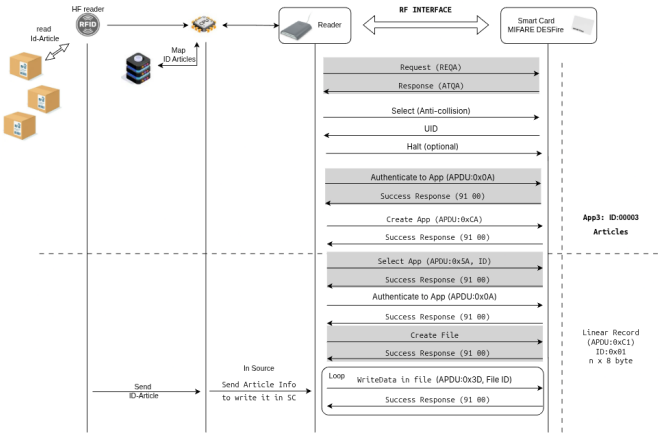

Fig. 10. Writing Articles in DESFire

smooth and seamless way of the Drivers, articles, vehicules and of cours the tags. The API follows all the basic best practices in its conception with well defined endpoints and well thought configuration. All the necessary CRUDs are defined in the swagger documentation that can be accessed through the API itself, with the correct inputs and what results to expect.